Hyperparameter tuning

Chọn tham số mà điểm báo cáo vẫn đáng tin

Một lần thử cần một câu hỏi

Đổi k có giảm overfit không?

Feature mới có sửa nhóm lỗi không?

Giữ nguyên split và metric để biết điểm đổi do điều gì.

Ghi lại cấu hình, không chỉ ghi điểm.

Mục tiêu

- Phân biệt tham số model học được với hyperparameter.
- Tính số lần huấn luyện của một lưới thử.
- Dùng GridSearchCV và RandomizedSearchCV với Pipeline.
- Chấm test một lần sau khi đã chọn xong.

01

Quy trình lựa chọn

Parameter và hyperparameter

Parameter: học từ train, như hệ số của linear regression.

Hyperparameter: ta chọn bằng validation, như , độ sâu, mức phạt, ngưỡng.

Chọn feature hay cách tiền xử lý cũng là lựa chọn như vậy.

HỎI NHANH

Ngưỡng phân lớp được chọn sao cho F1 trên validation cao nhất. Ngưỡng ấy có tính là một phần của phương án cần đánh giá không?

A Không, vì nó không được học như trọng số

B Có, vì nó được chọn bằng điểm validation

C Không, chỉ trọng số mới cần đánh giá

D Chỉ khi ngưỡng khác mặc định 0.5

Dev và test đã tách

KẾT QUẢ

```
Dev/test shape: (192, 6) (48, 6)
Số lớp 1 dev/test: 73 18
```

Mẫu độc lập, lớp 1 ít hơn: dùng stratified split.

Pipeline chứa StandardScaler.

Metric lựa chọn: F1 lớp 1.

Mọi fold fit cả Pipeline

```
pipeline = Pipeline([
    ('scale', StandardScaler()),
    ('knn', KNeighborsClassifier()),
])
```

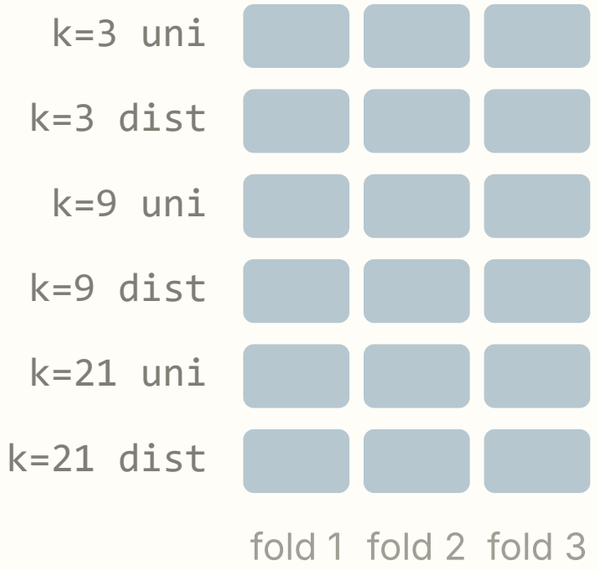
Scaler được fit riêng trong từng fold.

Tham số dùng tên `knn__n_neighbors`.

02

Ngân sách search

Đếm fit trước khi chạy



refit trên toàn dev

6 cấu hình × 3 fold

+ 1 refit

= 19 lần fit

Grid của bài: 6 cấu hình × 3 fold, cộng một lần refit, tổng 19 lần fit.

HỎI NHANH

Grid search thử bốn giá trị của A và ba giá trị của B, dùng cross-validation năm fold, rồi refit một lần. Tổng cộng phải fit bao nhiêu lần?

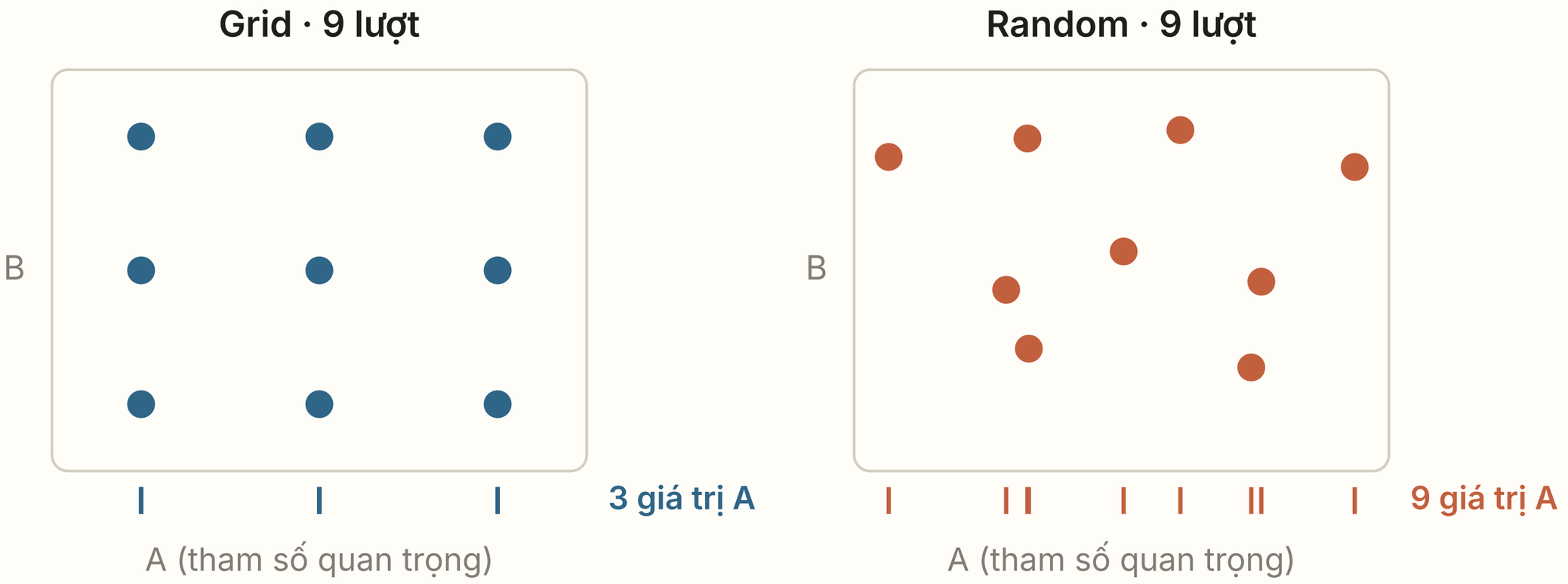
A 12

B 17

C 60

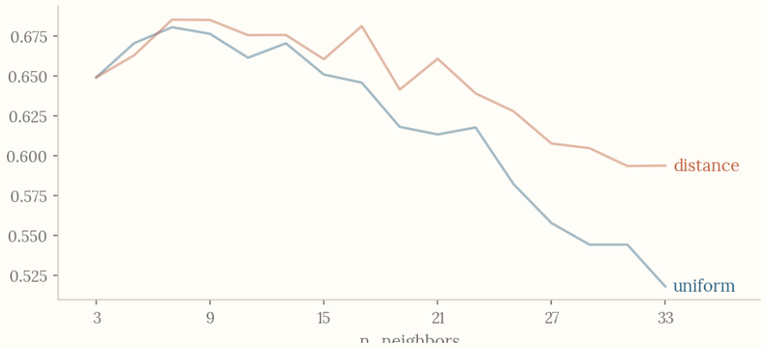
D 61

Grid và random

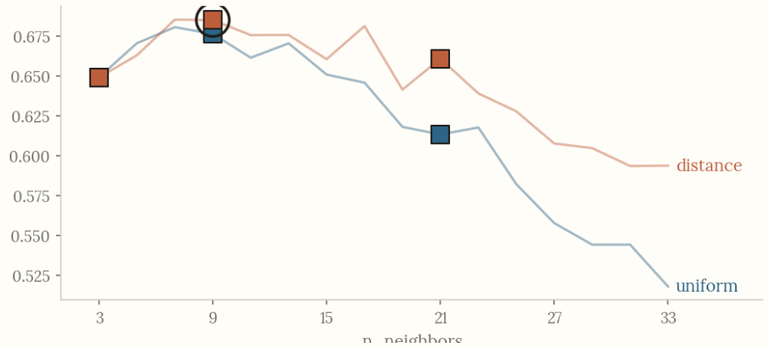


Cùng 9 lượt: grid chỉ thử 3 giá trị của tham số quan trọng, random thử 9.

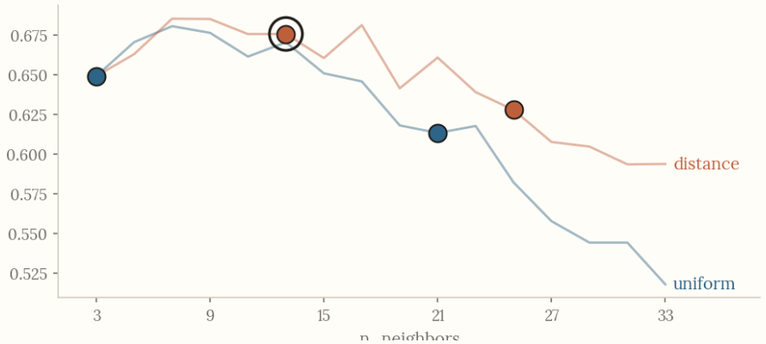
Grid và random trên cùng một bài



F1 cross-validation của mọi k lẻ từ 3 tới 33, hai kiểu weights (nét mờ, chỉ để nhìn; thường không có)



grid thử 6/6: k = 21, distance, F1 0.661; tốt nhất đến giờ k = 9, distance



random thử 4/4: k = 13, distance, F1 0.676; tốt nhất đến giờ k = 13, distance

Cùng pipeline và cross-validation của bài: grid search thử 6 tổ hợp cố định (ô vuông), random search bốc 4 tổ hợp (chấm tròn); vòng lớn là tổ hợp tốt nhất đã thấy.

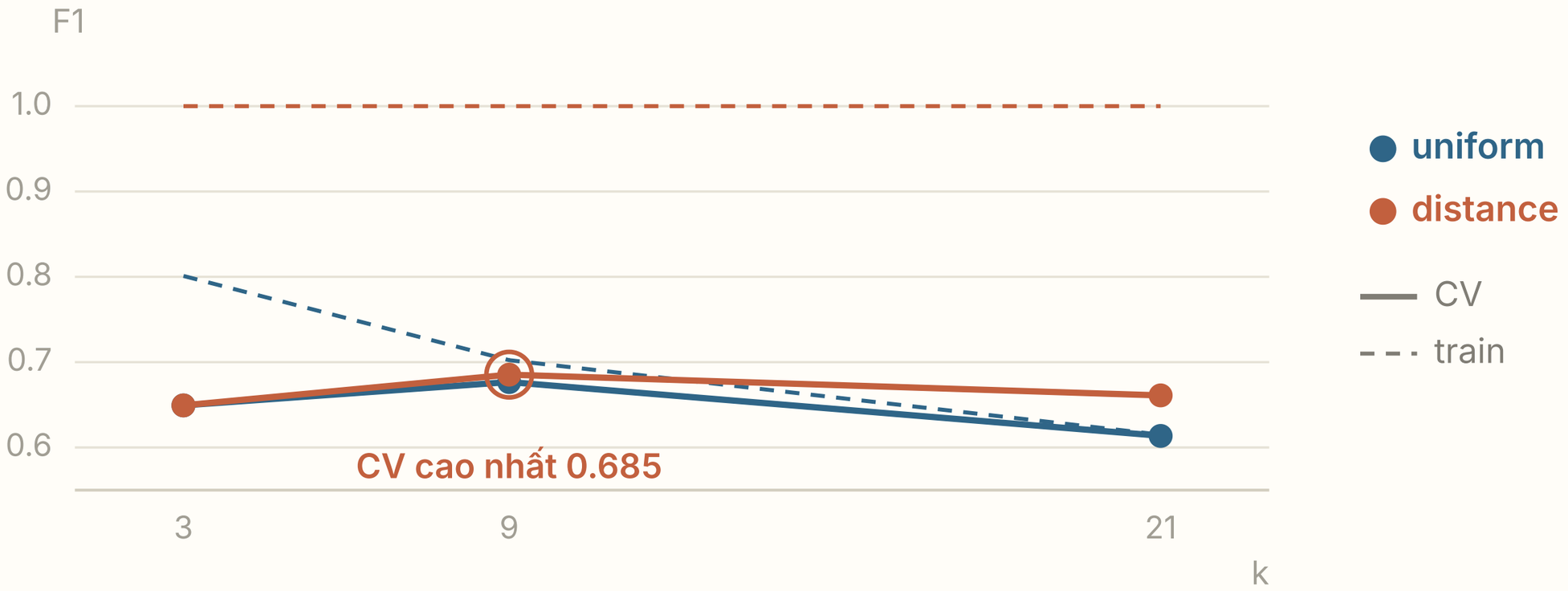
Grid đã chạy

KẾT QUẢ

```
Grid chọn: {'knn__n_neighbors': 9, 'knn__weights': 'distance'}
Best CV F1=0.685
```

, weights=distance, CV F1 0.685.

Bảng log thành hình



Nét liền là F1 trên CV, nét đứt là trên train; distance luôn khớp train 1.0.

Đọc log train và validation

Distance: train score có thể bằng 1 vì điểm là hàng xóm của chính nó.

`mean_test_score` trong CV = validation fold.

Không phải test ngoài.

Xem từng fold khi cải thiện nhỏ.

HỎI NHANH

Mọi dòng `weights=distance` đều có `mean_train_score` bằng 1.000. Vì sao vậy?

A Vì nó overfit nặng nhất

B Vì scaler lấy thông tin của validation

C Vì mỗi điểm train cách chính nó một khoảng 0

D Vì nó tốt nhất, điểm train cao nhất

Random đã chạy

KẾT QUẢ

```
Số ứng viên grid/random: 6 4
Random chọn: {'knn__weights': 'distance', 'knn__n_neighbors': 13}
Random best CV F1=0.676
```

4 ứng viên × 3 fold + 1 refit = 13 fit, cùng CV và metric với grid.

HỎI NHANH

Tuning xong, có thể báo `GridSearchCV.best_score_` như một điểm test độc lập không?

A Được, vì cột trong log tên là `mean_test_score`

B Được, nếu dùng từ năm fold trở lên

C Không, vì đó là điểm CV đã dùng để chọn

D Không, vì đó là điểm train của người thắng

03

Chốt và báo cáo

Khi hai ứng viên hòa

A: (0.7,0.8,0.9).

B: (0.8,0.8,0.8).

Cùng trung bình 0.8.

Quy tắc hòa đặt trước test.

Không tự suy ra B chắc chắn tổng quát tốt hơn.

Có thể xét chi phí hoặc độ ổn định trên dev.

Chọn bằng CV, chấm test

KẾT QUẢ

```
Phương án đã chốt: grid {'knn__n_neighbors': 9, 'knn__weights': 'distance'}
Test F1=0.727
[[27  3]
 [ 6 12]]
```

Grid thắng theo CV. `best_estimator_` đã refit trên toàn dev.

Test F1 0.727, đọc cả confusion matrix.

HỎI NHANH

Thấy F1 trên test thấp, cả đội chỉnh tiếp mô hình, rồi báo điểm trên chính test ấy như chưa từng dùng. Làm vậy có được không?

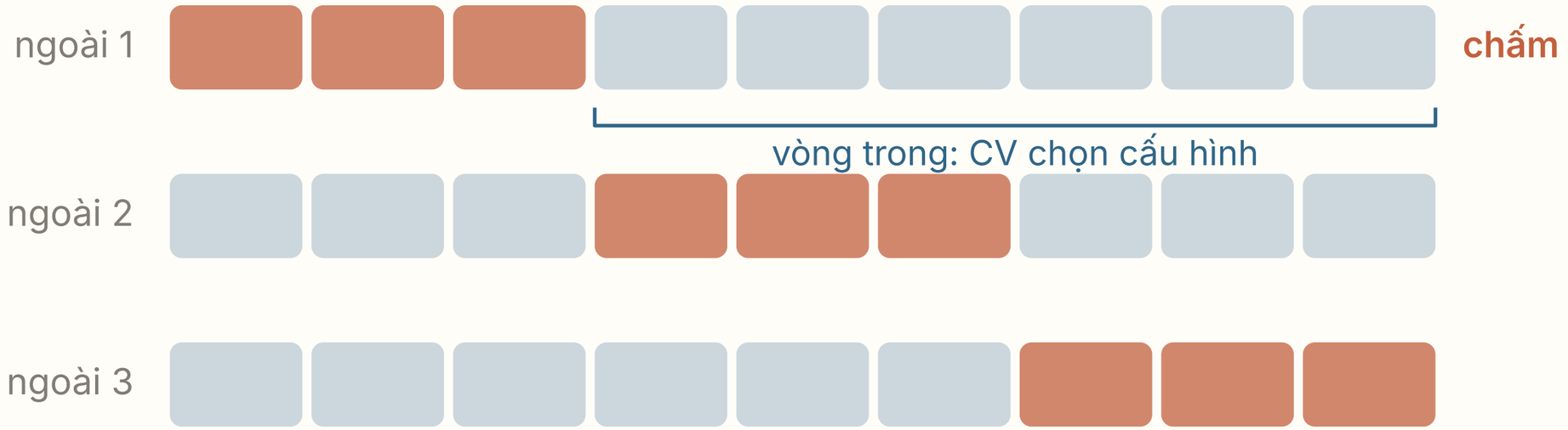
A Không, vì các lần chỉnh sau đã dựa vào test

B Được, nếu chỉ đổi vài tham số

C Được, miễn là không fit trên test

D Không, vì phải bỏ hẳn test đi

Nested CV khi cần



Nested CV: phần chấm ở vòng ngoài không tham gia lựa chọn nào.

Cái giá của nested CV

Vòng trong chọn cấu hình; vòng ngoài chấm toàn quy trình chọn.

Tốn thêm fit.

Best CV score của vòng trong thường lạc quan sau chọn.

Test ngoài là phương án đơn giản nếu có đủ dữ liệu.

Log và bước thử tiếp

Ghi lại phiên bản dữ liệu, cách chia, feature, model, tham số, seed, metric và thời gian.

Chọn giả thuyết tiếp theo từ các lỗi còn lại.

Dò seed hay bảng điểm công khai cho tới khi đẹp không phải là đánh giá độc lập.

Khi đi thi

- Số fit = tổ hợp × fold (+1 refit).
- Mỗi lần thử đổi một ý; ghi tham số, seed, điểm CV.
- Chọn bài cuối theo CV, không theo bảng công khai.

04

Luyện đề

LUYỆN ĐỀ 1

Mỗi lần train mất 1 giờ, dữ liệu lớn hay nhỏ cũng vậy. Có 4 lượt chia, mỗi lượt 25% làm validation. có 8 giá trị, và ta có 100 giờ. được thử tối đa mấy giá trị?

A 12

B 3

C 4

D 25

PHỎNG THEO CMU 10-601 · Exam 1 Practice · Q3.3

LUYỆN ĐỀ 2

Một đội nộp 60 lần, lần nào cũng giữ bản có điểm công khai (tính trên 30% test) cao hơn. Điểm công khai tăng từ 0.712 lên 0.741, còn điểm CV chỉ quanh 0.70. Điểm bảng cuối nhiều khả năng là bao nhiêu?

A Gần 0.70 hơn là gần 0.741

B Khoảng 0.741, vì hai phần test giống nhau

C Cao hơn 0.741, vì phần bảng cuối lớn hơn

D Không đoán được

Chốt ý

Tuning cũng là học: chọn bằng CV trên dev, không bằng test.

Giữ nguyên Pipeline, split và metric cho mọi ứng viên.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook: đếm số fit trước, rồi chạy grid và so với con số đã đếm.